

COMS W4156 Advanced Software Engineering (ASE)

September 10, 2024

On this day ...



Africa's 1st Gold Medal

1960 [Abebe Bikila](#) of Ethiopia runs WR 2:15:16.2 barefoot to take out the men's marathon at the Rome Olympics; Africa's first ever Olympic gold medal

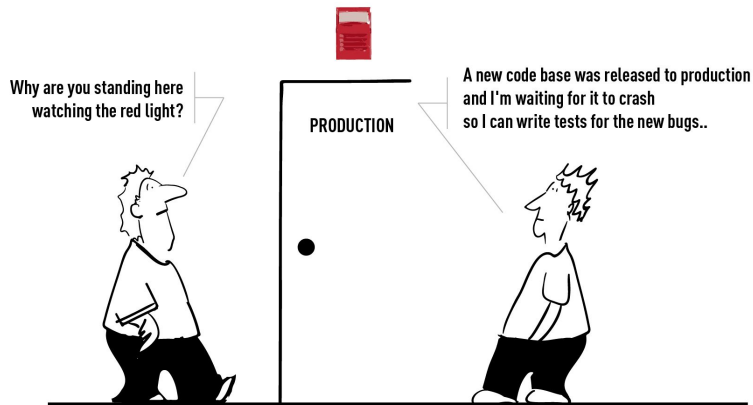


Marathon Runner

[Abebe Bikila](#)

Agenda

1. Bug Finding Whirlwind Tour
2. Java/C++ bug finding demo: Shreya Kochar and Shivam Chaturvedi
3. Upcoming

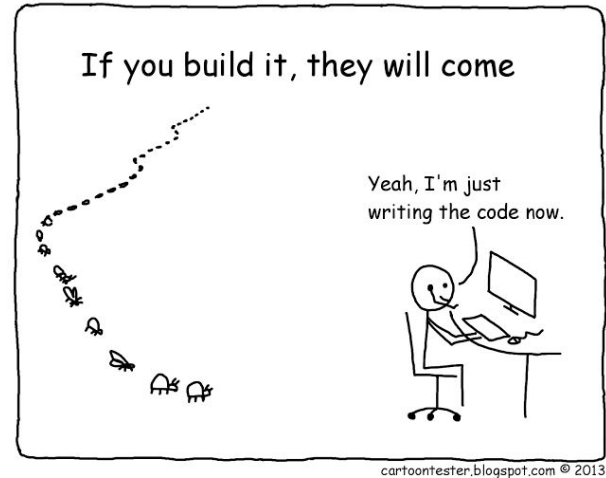


Bugs Are Everywhere

Software has bugs. This is normal.

Buggy code does not matter much if it never leaves the developer's machine

Buggy code committed to shared repositories will increase time and effort for *other* software engineers on the same team, but unlikely to be humongously expensive if bugs never make it to production.



Preventing Buggy Code from being Released to Production



Sad & Useless Humor
@sadanduseless

ChatGPT programmer meme.

THE #1 PROGRAMMER EXCUSE
FOR LEGITIMATELY SLACKING OFF:
"CHATGPT IS DOWN"



2:09 PM · Jun 13, 2023

Human Code Review is labor-intensive and limited by human patience

AI Code Review is promising, but limited by context length and "hallucinations"

What can be Automated *at Scale*?

"Finally we have agents that work at least 31.76% of the time!"



Mike Walker
Senior Agent Engineer

Automatically Finding Bugs *at Scale*

Examine the code: Static Analysis

Execute the code: Dynamic Analysis



Static Analysis



Any program analysis technique that does not actually execute the code is "static" (program analysis techniques that do execute the code are "dynamic")

Most static analyzers do not need to know anything about what your code is supposed to do, they look for generic problems that occur in many programs

Includes [style checkers](#): non-compliance with coding standards does not necessarily mean there's a bug, but does make it more likely for bugs to occur as multiple developers (mis-)understand and modify the code.

But the term static analysis or static analyzer more commonly refers to "code smell detectors" and "bug finders", often combined, like [Infer](#) and [SonarCloud](#)

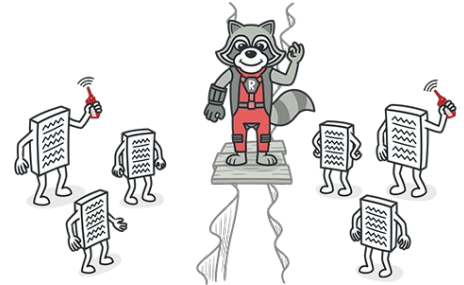
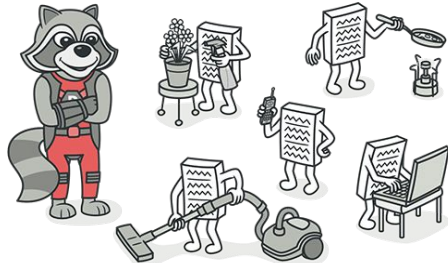
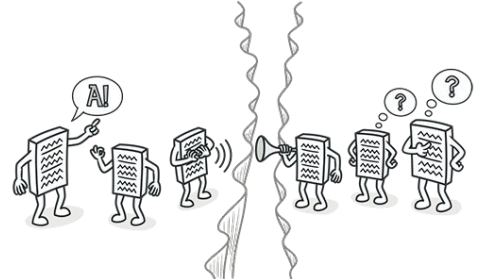
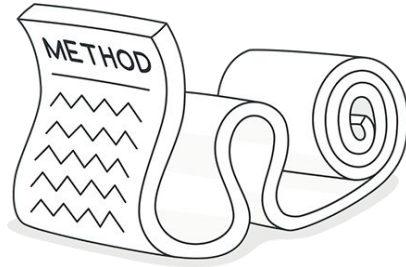
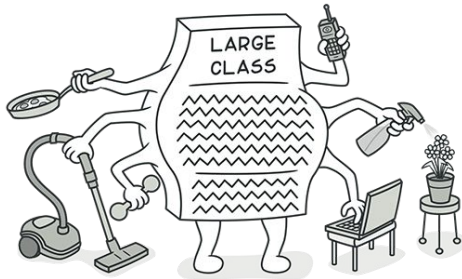
Code Smells

A "[code smell](#)" is a surface indication in the program text that usually corresponds to a deeper problem in the codebase

Like poor coding style, code smells do not necessarily indicate a bug, yet, but tend to make changes more difficult and more likely to lead to bugs as multiple developers (or your future self) modify the code



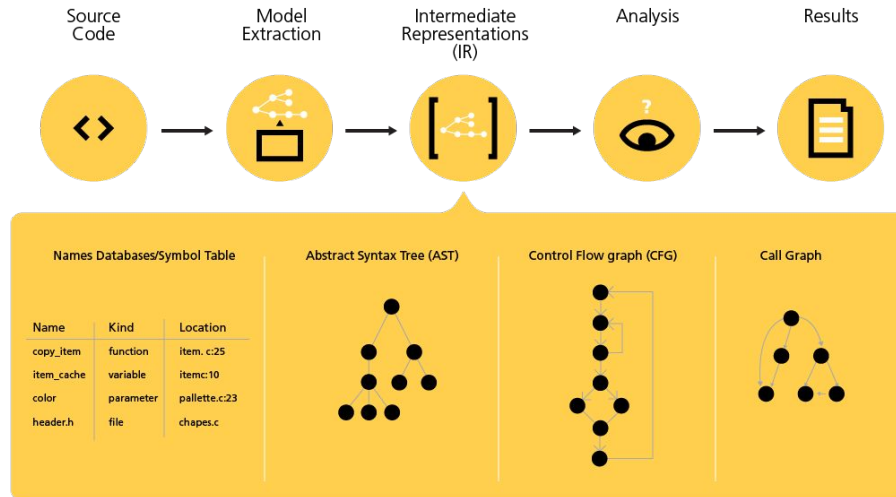
Example Code Smells: Bloaters and Couplers



Bug Finders

Style checkers usually work by scanning the code as text

Code smell detectors and bug finders instead look for "patterns" in an intermediate representation of code



Some Suspicious Bug Patterns That Can Be Detected Staticly

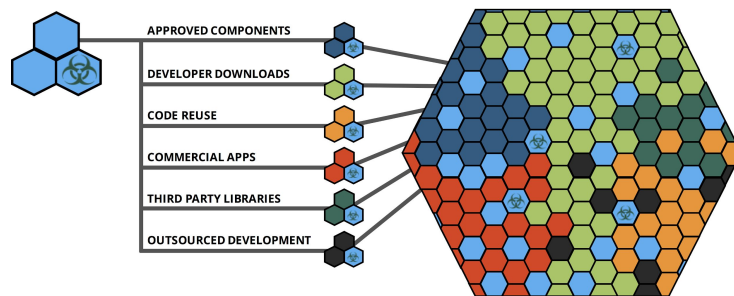
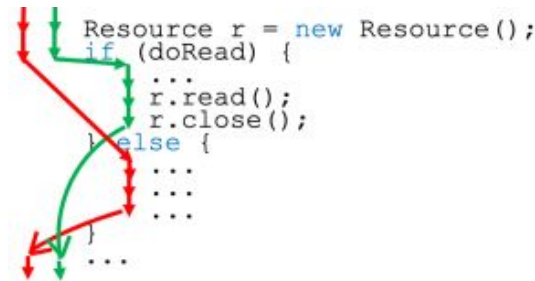
Resource leaks where resources (e.g., memory, locks, database connections) are not freed on every code path reachable from where they were allocated

Using a "blacklisted" library, service, framework with known security vulnerabilities (usually an older version)

Code includes "secrets" for accessing cloud infrastructure (not technically a bug, but akin to telling the world the userid/passwd for accounts you pay for)

Others...

```
Resource r = new Resource();
if (doRead) {
    ...
    r.read();
    r.close();
} else {
    ...
    ...
}
```



```
Tree: 6d33b... / aws / claudia / server.js
aws_exercises
1 contributor
21 lines (15 sloc) | 361 Bytes
1 var ApiBuilder = require('claudia-api-builder'),
2   api = new ApiBuilder();
3
4 module.exports = api;
5
6 AWS.config.update({
7   "accessKeyId": "AKIA...",
8   "secretAccessKey": "..."
9 })
10 ...
```

What Kinds of Bugs Can Be Found By Executing the Code?



The tester (or test script) enters a number, presses +, enters another number, presses =

- Nothing happens
- Computes the wrong answer

Calculator works correctly for some period of time or some number of computations, and then does nothing

Calculator displays all 0's and won't do anything else

The calculator correctly adds, subtracts, multiplies and divides, but the tester found that if two operators are held down simultaneously, it appears to do square root

The calculator's buttons are too small, the = key is in an odd place, the display is hard to read, ...

Is This a Bug or a Feature?

The tester (or test script) enters a number, presses +, enters another number, presses =

- Nothing happens
- Computes the wrong answer

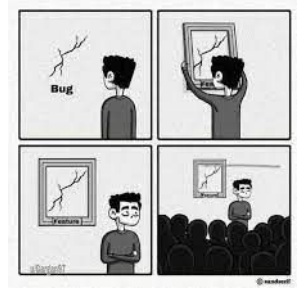
Calculator works correctly for some period of time or some number of computations, and then does nothing

Calculator displays all 0's and won't do anything else

The calculator correctly adds, subtracts, multiplies and divides, but the tester found that if two operators are held down simultaneously, it appears to do square root

The calculator's buttons are too small, the = key is in an odd place, the display is hard to read, ...

Probably a bug



Bug - memory or other resource leak;
Feature - trial period over, now you have to pay for it

Bug - code stuck in infinite loop; Feature - battery low

Undocumented feature. '[Exploratory testing](#)' = "what happens when I do this?"

[Usability bug](#). The user will consider these bugs even though the developers may not

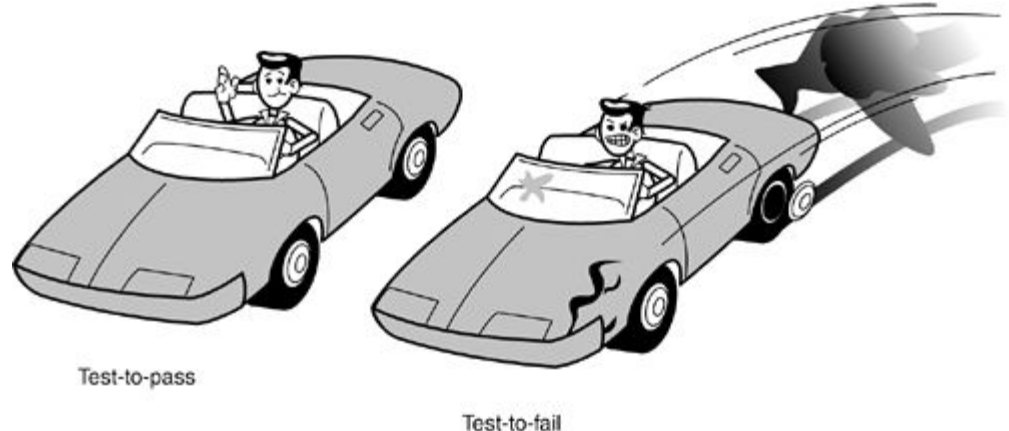
Test-to-Pass vs. Test-to-Fail

Initial testing makes sure the software minimally works with typical valid inputs ("[smoke testing](#)")

At first try only the simplest and most straightforward test cases ("[happy path](#)")

Most testing is intended to reveal bugs, so they can be fixed

Test with inputs designed to trigger "[corner cases](#)", e.g., try to save a file to a flash drive when there isn't one inserted



How to Choose Test Inputs

Typical valid inputs

Enter age: _____25_____

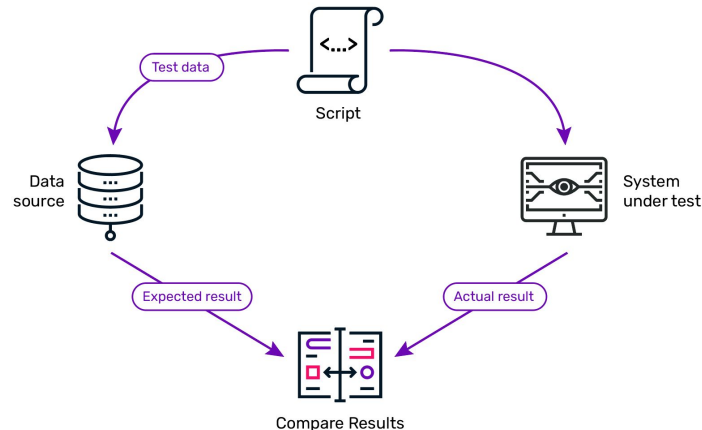
Atypical valid inputs

Enter age: _____116_____

Invalid inputs

Enter age: _____3.14159_____

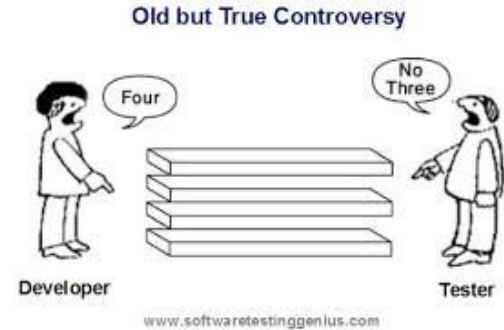
More sophisticated
approaches to choosing
test inputs introduced
later in course



Invalid Input Formats

String inputs with valid vs. invalid *formatting*

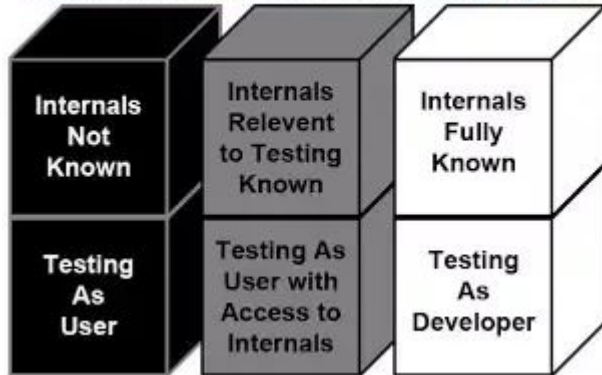
- [UTF-8](#) when [ASCII](#) is required (and vice versa)
- Printing vs. non-printing characters (e.g., control characters)
- Upper vs. lower case
- Space, tab, newline characters (whitespace)
- Any characters with special meaning to the application (or to the implementation, e.g., programming language, SQL)
- Null/empty string
- ...



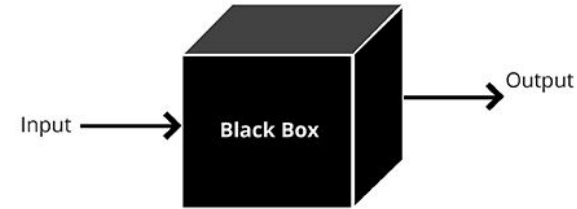
Approaches To Testing

Black box - tester considers what the box is supposed to do, not how it does it, doesn't look inside the box

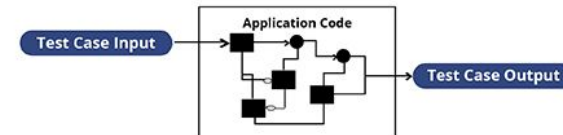
Differences Between Box Testing Types



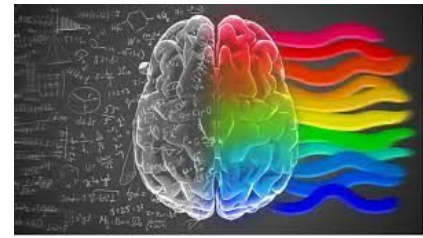
BLACK BOX TESTING APPROACH



WHITE BOX TESTING APPROACH



Testing is not very Colorful



Gray box (or grey box) - tester looks at intermediate products between boxes (e.g., network I/O) and leftover side-effects after box is done executing (e.g., temporary files, database connections left open)

White box - tester leverages full knowledge of what is inside the box

- If you never ran this part of the code, how can you have any confidence that it works?

Coverage

At minimum, testing should exercise every statement

Better: Exercise every branch

Beware the missing else!

100% coverage may check missing conditional cases but can never detect entirely missing code



```
if (condition) { do something }  
else { do something else }
```

```
if (condition) { do something }
```

More More More

Most of the semester will be about finding and fixing bugs

All these testing topics, and others, will be discussed more extensively later



Agenda

1. Bug Finding Whirlwind Tour
2. Java/C++ bug finding demo: Shreya Kochar and Shivam Chaturvedi
3. Upcoming



Shreya Kochar and Shivam Chaturvedi Java/C++ Bug Finding Demo

[demo slides](#)



Agenda

1. Bug Finding Whirlwind Tour
2. Java/C++ bug finding demo: Shreya Kochar and Shivam Chaturvedi
3. **Upcoming**



Deadline Reminders

[Individual Miniproject part 1](#) due Friday September 13 11:59pm

[Part 2](#) due September 20, [Part 3](#) due September 27



Planning Ahead to Team Project

Team Formation due September 20

Ideally 4 students per team, 5 students ok

Team Project Proposal due
September 27

No teams of 1, 2, 3, 6, 7, ...

(Note these deadlines coincide with
Miniproject deadlines, do not wait!)

Sanity Checks:

- Everyone fluent in team's choice of Java or C++
- Everyone has same development platform - Linux, Mac, Windows
- Everyone has compatible schedules
- ***Everyone is enrolled in this course and does not plan to drop***

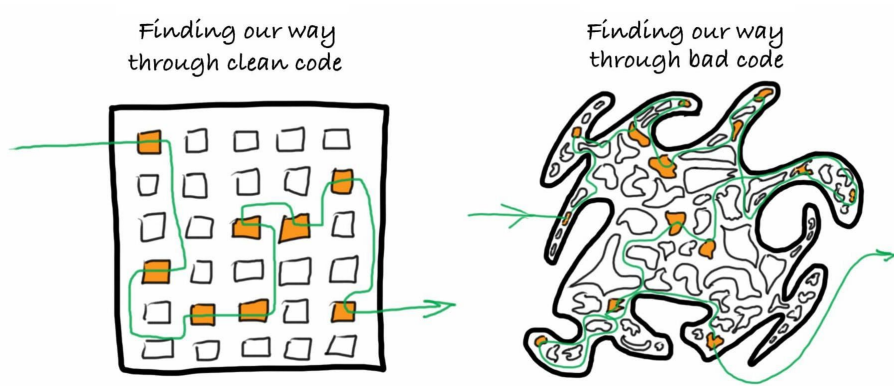


Thursday

Finish anything we didn't get to today

Libraries and Services

"Clean Code" demo





Ask Me Anything

